

ThinkOn Accounting — MVP Backend Build Spec

Everything the backend developer needs to build Phases 0–5 without re-reading the full 37-section document: real table schemas, the posting engine algorithm, validation rules, and the exact API surface for launch.

Does it have a Chart of Accounts? Yes. The full spec reverse-engineers all 142 accounts from your source data with hierarchy rules and a ready schema — it's Phase 0 below, in full.

SCOPE & GROUND RULES

Read this before writing any code

This is the trimmed MVP cut of a much larger spec (37 sections, 8 build phases). Building all of it is not a 3-month, 2-backend-dev job. This document is **self-contained for Phases 0–5** — the schema and rules below are copy-ready. The full PDF is still the reference for edge cases and anything marked deferred at the end.

THE ONE ARCHITECTURAL RULE THAT GOVERNS EVERYTHING BELOW

Modules never write to the GL directly. Sales, Purchasing, and every future module emit an accounting event; a single `posting engine` (Phase 1) turns that into balanced journal lines using configurable `posting_rule` rows — never hardcoded account numbers in module code. This is what lets you add tax, branches, or currencies later without touching Sales/Purchasing code.

BUILD THESE — IN ORDER

0 Foundation — Company, Branch & Chart of Accounts

Every other table points back to a company, branch, fiscal period, or GL account. Get this shape right before a single transaction exists — changing it later means migrating live journal data.

BUILD ORDER

1. Seed `account_category` with the **7-category model** below — not the raw COA CSV's 5-category column. Importing that column directly misclassifies every expense account.
2. Extend `account_sub_category` ranges to cover: Equity Distributions 3300–3599, Other COGS 5400–5599, Other Non-Operating 8200–8599 — before seeding accounts into them.
3. Import all 142 accounts into `gl_account`. Resolve `parent_account_id` strictly from the source's *Parent Account* column — never infer it from numeric code prefixes (e.g. assuming 1110 is a child of 1100).
4. Seed the 4 MVP-relevant GAP-FIX accounts now (table below) — cheap to add now, a migration later.
5. Create one `company` row, one `branch` row, one `fiscal_year` with 12 `fiscal_period` rows, all `OPEN`.

GL_ACCOUNT.SQL

```
CREATE TABLE account_category (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  code VARCHAR(20) NOT NULL UNIQUE,      -- ASSETS, LIABILITIES, EQUITY, REVENUE, COGS, OPEX, OTHER_EXPENSE  
  name VARCHAR(100) NOT NULL,  
  code_range_low INT NOT NULL,
```

```

code_range_high INT NOT NULL,
normal_balance CHAR(1) NOT NULL CHECK (normal_balance IN ('D','C')),
financial_statement VARCHAR(20) NOT NULL CHECK (financial_statement IN ('BALANCE_SHEET','INCOME_STATEMENT')),
display_order INT NOT NULL
);

CREATE TABLE account_sub_category (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  code VARCHAR(30) NOT NULL UNIQUE,
  name VARCHAR(100) NOT NULL,
  parent_category_id UUID NOT NULL REFERENCES account_category(id),
  code_range_low INT NOT NULL,
  code_range_high INT NOT NULL
);

CREATE TABLE gl_account (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES company(id),
  account_code VARCHAR(20) NOT NULL,
  account_name VARCHAR(150) NOT NULL,
  parent_account_id UUID REFERENCES gl_account(id),      -- explicit adjacency list, never derived from code
  category_id UUID NOT NULL REFERENCES account_category(id),
  sub_category_id UUID REFERENCES account_sub_category(id), -- nullable, never silently dropped from reports
  account_type VARCHAR(10) NOT NULL CHECK (account_type IN ('HEADER','DETAIL')),
  normal_balance CHAR(1) NOT NULL CHECK (normal_balance IN ('D','C')),
  is_contra BOOLEAN NOT NULL DEFAULT FALSE,
  is_control_account BOOLEAN NOT NULL DEFAULT FALSE,      -- TRUE for AR (1120), AP (2110)
  control_account_type VARCHAR(20),                      -- 'AR','AP', null otherwise
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  is_postable BOOLEAN GENERATED ALWAYS AS (account_type = 'DETAIL' AND is_active) STORED,
  description VARCHAR(255),
  UNIQUE (company_id, account_code)
);
CREATE INDEX idx_gl_account_parent ON gl_account(parent_account_id);
CREATE INDEX idx_gl_account_control ON gl_account(company_id, control_account_type) WHERE is_control_account = TRUE;

CREATE TABLE company (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  legal_name VARCHAR(200) NOT NULL,
  functional_currency CHAR(3) NOT NULL,
  fiscal_year_start_month INT NOT NULL DEFAULT 1,
  country_code CHAR(2) NOT NULL
);

CREATE TABLE branch (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES company(id),
  branch_code VARCHAR(20) NOT NULL,
  branch_name VARCHAR(150) NOT NULL,
  is_head_office BOOLEAN NOT NULL DEFAULT FALSE,
  UNIQUE (company_id, branch_code)
);

CREATE TABLE fiscal_year (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES company(id),
  fiscal_year INT NOT NULL,
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  status VARCHAR(15) NOT NULL DEFAULT 'OPEN' CHECK (status IN ('OPEN','SOFT_CLOSE','HARD_CLOSE')),
  UNIQUE (company_id, fiscal_year)
);

CREATE TABLE fiscal_period (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  fiscal_year_id UUID NOT NULL REFERENCES fiscal_year(id),
  period_number INT NOT NULL,
  start_date DATE NOT NULL,
  end_date DATE NOT NULL,
  status VARCHAR(15) NOT NULL DEFAULT 'OPEN' CHECK (status IN ('OPEN','SOFT_CLOSE','HARD_CLOSE')),
  UNIQUE (fiscal_year_id, period_number)
);

```

MVP GAP-FIX ACCOUNTS TO SEED NOW

CODE	NAME	PARENT	WHY
1115	Undeposited Funds	1110	Customer payments received before bank deposit clears — posting straight to Checking breaks reconciliation

2155	GRNI (Goods Received Not Invoiced)	2100	Clears the timing gap between goods receipt and vendor bill
1145	Input Tax Receivable	1100	Keeps the schema VAT-ready even if only one non-recoverable rate is active at launch
2185	Deferred Revenue	2100	Only needed if any product is subscription/multi-period — skip if not

COMMON MISTAKE

Deriving hierarchy from code ranges ("1110 must be a child of 1100"). Section 1.1 of the full spec proves this is false in the real data — always read `parent_account_id` from the explicit relationship.

DONE WHEN

All 142 accounts import with zero orphaned parents; every Header account has children, every Detail account has none; a recursive query rolls any account up to its category.

1 Posting Engine — The Double-Entry Core

Everything else writes through this. Get the balance-check and immutability guarantees right here and every module built on top inherits them for free.

JOURNAL_ENTRY.SQL

```
CREATE TABLE journal_entry (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES company(id),
  journal_number VARCHAR(30) NOT NULL,      -- JE-2026-000123
  journal_type VARCHAR(20) NOT NULL CHECK (journal_type IN ('STANDARD','REVERSING','OPENING_BALANCE')),
  source_module VARCHAR(30) NOT NULL,      -- 'AR','AP','MANUAL'
  source_document_type VARCHAR(30),
  source_document_id UUID,
  fiscal_period_id UUID NOT NULL REFERENCES fiscal_period(id),
  entry_date DATE NOT NULL,
  currency_code CHAR(3) NOT NULL,
  description VARCHAR(500),
  status VARCHAR(15) NOT NULL DEFAULT 'DRAFT' CHECK (status IN ('DRAFT','PENDING_APPROVAL','POSTED','REVERSED')),
  reversal_of_entry_id UUID REFERENCES journal_entry(id),
  reversed_by_entry_id UUID REFERENCES journal_entry(id),
  created_by UUID NOT NULL REFERENCES app_user(id),
  approved_by UUID REFERENCES app_user(id),
  posted_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (company_id, journal_number)
);

CREATE TABLE journal_line (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  journal_entry_id UUID NOT NULL REFERENCES journal_entry(id) ON DELETE RESTRICT,
  line_number INT NOT NULL,
  account_id UUID NOT NULL REFERENCES gl_account(id),
  debit_amount DECIMAL(19,4) NOT NULL DEFAULT 0 CHECK (debit_amount >= 0),
  credit_amount DECIMAL(19,4) NOT NULL DEFAULT 0 CHECK (credit_amount >= 0),
  customer_id UUID REFERENCES customer(id),  -- populated only when account is a control account
  vendor_id UUID REFERENCES vendor(id),
  memo VARCHAR(255),
  CONSTRAINT chk_one_sided CHECK (
    (debit_amount > 0 AND credit_amount = 0) OR (credit_amount > 0 AND debit_amount = 0)
  ),
  UNIQUE (journal_entry_id, line_number)
);

CREATE INDEX idx_journal_line_account ON journal_line(account_id);

-- posting_rule / posting_rule_line: how modules stay decoupled from hardcoded GL numbers
CREATE TABLE posting_rule (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES company(id),
  rule_code VARCHAR(50) NOT NULL,      -- 'AR_INVOICE','AP_BILL','INV_RECEIPT'
  source_module VARCHAR(30) NOT NULL,
  UNIQUE (company_id, rule_code)
);

CREATE TABLE posting_rule_line (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

posting_rule_id UUID NOT NULL REFERENCES posting_rule(id),
line_number INT NOT NULL,
account_source VARCHAR(30) NOT NULL,    -- 'FIXED' | 'CUSTOMER.AR_ACCOUNT' | 'PRODUCT.REVENUE_ACCOUNT'
fixed_account_id UUID REFERENCES gl_account(id),
side CHAR(1) NOT NULL CHECK (side IN ('D','C')),
amount_source VARCHAR(30) NOT NULL      -- 'LINE_AMOUNT' | 'TAX_AMOUNT'
);

-- period-cached balance, read by Trial Balance instead of summing raw journal_line every time
CREATE TABLE account_balance (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  account_id UUID NOT NULL REFERENCES gl_account(id),
  fiscal_period_id UUID NOT NULL REFERENCES fiscal_period(id),
  opening_balance DECIMAL(19,4) NOT NULL DEFAULT 0,
  period_debit_total DECIMAL(19,4) NOT NULL DEFAULT 0,
  period_credit_total DECIMAL(19,4) NOT NULL DEFAULT 0,
  closing_balance DECIMAL(19,4) NOT NULL DEFAULT 0,
  UNIQUE (account_id, fiscal_period_id)
);

```

POSTING ENGINE ALGORITHM

```

function post(accountingEvent):
  rule = lookupPostingRule(event.sourceModule, event.eventType)
  lines = []
  for template in rule.lines:
    account = resolveAccount(template.account_source, event)
    amount = resolveAmount(template.amount_source, event)
    validateAccountIsPostable(account)    -- is_postable = TRUE, reject Header accounts
    validateAccountIsActive(account)
    validateFiscalPeriodOpen(event.period)
    lines.append({account, side: template.side, amount})

  assert sum(debits) == sum(credits)      -- hard fail if the rule itself is misconfigured
                                          -- NEVER auto-balance into a suspense account

  BEGIN TRANSACTION
    journalEntry = createJournalEntry(status='DRAFT')
    for line in lines: createJournalLine(journalEntry, line)
    journalEntry.status = 'POSTED'        -- or PENDING_APPROVAL, see Phase 4
    updateAccountBalanceCache(...)
    updateSubledger(...)                  -- same transaction, atomic with the GL post
  COMMIT

```

NON-NEGOTIABLE RULES FOR THIS PHASE

- **Balance check is a deferred DB trigger**, not an app-layer check — `sum(debits) = sum(credits)` per `journal_entry_id`, enforced at commit so a bug or a bad migration can never persist an unbalanced entry.
- **Posted entries are immutable** — a `BEFORE UPDATE/DELETE` trigger on `journal_line` for any row whose parent is `POSTED`. Corrections happen only via reversal, never edits.
- **Debit and credit are separate non-negative columns**, never one signed amount — this is deliberate, not stylistic (needed for the one-sided constraint and for Trial Balance presentation).

DONE WHEN

A manual journal entry posts, balances, and shows correctly in a Trial Balance; attempting to edit a posted entry is rejected by the database itself.

2AR / AP Subledgers

One GL account each for AR (1120) and AP (2110) — never one GL account per customer. Customer/vendor-level detail lives in the subledger, which must always sum to the control account.

SUBLEDGER.SQL

```
CREATE TABLE customer (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES company(id),
  customer_code VARCHAR(30) NOT NULL,
  name VARCHAR(200) NOT NULL,
  ar_control_account_id UUID NOT NULL REFERENCES gl_account(id), -- normally 1120
  default_currency_code CHAR(3) NOT NULL,
  payment_terms_days INT NOT NULL DEFAULT 30,
  UNIQUE (company_id, customer_code)
);

CREATE TABLE vendor (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES company(id),
  vendor_code VARCHAR(30) NOT NULL,
  name VARCHAR(200) NOT NULL,
  ap_control_account_id UUID NOT NULL REFERENCES gl_account(id), -- normally 2110
  default_currency_code CHAR(3) NOT NULL,
  payment_terms_days INT NOT NULL DEFAULT 30,
  UNIQUE (company_id, vendor_code)
);

CREATE TABLE ar_subledger_transaction (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID NOT NULL REFERENCES customer(id),
  journal_line_id UUID NOT NULL REFERENCES journal_line(id),
  transaction_type VARCHAR(20) NOT NULL CHECK (transaction_type IN ('INVOICE','PAYMENT','WRITE_OFF')),
  transaction_date DATE NOT NULL,
  due_date DATE,
  amount DECIMAL(19,4) NOT NULL, -- signed: + increases AR, - decreases AR
  open_amount DECIMAL(19,4) NOT NULL -- remaining unapplied amount, drives aging
);

CREATE TABLE ar_cash_application (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  payment_transaction_id UUID NOT NULL REFERENCES ar_subledger_transaction(id),
  invoice_transaction_id UUID NOT NULL REFERENCES ar_subledger_transaction(id),
  applied_amount DECIMAL(19,4) NOT NULL CHECK (applied_amount > 0),
  applied_date DATE NOT NULL
);
-- ap_subledger_transaction / ap_cash_application mirror the AR tables exactly
```

RECONCILIATION INVARIANT — CHECK THIS CONTINUOUSLY

`SUM(journal_line.debit - credit) WHERE account = AR control account` must always equal

`SUM(ar_subledger_transaction.amount)`. Run this as a nightly job. If it ever drifts, something posted to the control account without writing a subledger row — never allow direct inserts to a control account's `journal_line` from outside the posting engine.

DONE WHEN

An invoice posts to the GL and the customer's subledger atomically; AR aging (30/60/90) works off `open_amount`, maintained transactionally as payments apply — never recomputed from scratch at report time.

3 Financial Statements

Trial Balance, Balance Sheet, Income Statement — all built on one filtered view so drafts and reversed entries can never leak into a report.

REPORTING_VIEWS.SQL

```
-- Every report reads through this view, never raw journal_line
CREATE VIEW gl_posted_lines AS
SELECT jl.*, je.entry_date, je.fiscal_period_id, je.company_id
FROM journal_line jl
JOIN journal_entry je ON je.id = jl.journal_entry_id
WHERE je.status = 'POSTED';

CREATE VIEW trial_balance AS
SELECT ga.account_code, ga.account_name, ac.normal_balance,
       SUM(jl.debit_amount) AS total_debits,
       SUM(jl.credit_amount) AS total_credits,
       SUM(jl.debit_amount) - SUM(jl.credit_amount) AS net_balance
FROM gl_account ga
JOIN account_category ac ON ac.id = ga.category_id
JOIN gl_posted_lines jl ON jl.account_id = ga.id
WHERE jl.entry_date <= :as_of_date
GROUP BY ga.account_code, ga.account_name, ac.normal_balance;

-- Balance Sheet: cumulative since inception
CREATE VIEW balance_sheet_line AS
SELECT ga.account_name, ac.code AS category_code,
       SUM(CASE WHEN ac.normal_balance = 'D' THEN jl.debit_amount - jl.credit_amount
                ELSE jl.credit_amount - jl.debit_amount END) AS balance
FROM gl_account ga
JOIN account_category ac ON ac.id = ga.category_id
JOIN gl_posted_lines jl ON jl.account_id = ga.id
WHERE ac.financial_statement = 'BALANCE_SHEET' AND jl.entry_date <= :as_of_date
GROUP BY ga.account_name, ac.code;

-- Income Statement: period-bound, resets each fiscal year
CREATE VIEW income_statement_line AS
SELECT ga.account_name, ac.code AS category_code,
       SUM(CASE WHEN ac.normal_balance = 'D' THEN jl.debit_amount - jl.credit_amount
                ELSE jl.credit_amount - jl.debit_amount END) AS balance
FROM gl_account ga
JOIN account_category ac ON ac.id = ga.category_id
JOIN gl_posted_lines jl ON jl.account_id = ga.id
WHERE ac.financial_statement = 'INCOME_STATEMENT' AND jl.entry_date BETWEEN :period_start AND :period_end
GROUP BY ga.account_name, ac.code;
```

MANDATORY ASSERTION

Trial Balance must hard-fail (not render) if $\text{SUM}(\text{total_debits}) \neq \text{SUM}(\text{total_credits})$ across all rows. An out-of-balance Trial Balance means system-level data corruption, not a normal state — never display it.

DONE WHEN

All three statements generate from real posted data and tie out exactly.

4 Minimal Controls

Trimmed hard from the full spec — enough to be safe for launch, not the full configurable engine.

AUDIT_LOG.SQL

```
CREATE TABLE audit_log (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  company_id UUID NOT NULL REFERENCES company(id),  
  table_name VARCHAR(50) NOT NULL,  
  record_id UUID NOT NULL,  
  action VARCHAR(10) NOT NULL CHECK (action IN ('INSERT','UPDATE','DELETE')),  
  changed_by UUID NOT NULL REFERENCES app_user(id),  
  changed_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  old_values JSONB,  
  new_values JSONB  
);  
CREATE INDEX idx_audit_log_record ON audit_log(table_name, record_id);  
-- Populate via a DB trigger on gl_account (and any other sensitive table) — never from app code only,  
-- so the log can't be bypassed by a direct write or a future code path that forgets to log.
```

TRIMMED FOR MVP

Skip the full configurable `approval_workflow/approval_request` engine. Hardcode one rule in the approval step:
`journal_entry.created_by != approver_id`, checked server-side, not just hidden in the UI. Upgrade to a configurable rule table later — nothing about this choice blocks that upgrade.

DONE WHEN

A self-approval attempt is rejected server-side; every `gl_account` edit is logged automatically with old/new values.

5Sales, Purchasing & Tax

Real invoices and bills flowing through the Phase 1 engine — trimmed to what launch actually needs.

TAX_ENGINE.SQL

```
CREATE TABLE tax_jurisdiction (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(100) NOT NULL,
  country_code CHAR(2) NOT NULL,
  tax_type VARCHAR(15) NOT NULL CHECK (tax_type IN ('SALES_TAX','VAT','GST','WITHHOLDING'))
);

CREATE TABLE tax_rate (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tax_jurisdiction_id UUID NOT NULL REFERENCES tax_jurisdiction(id),
  rate_percent DECIMAL(7,4) NOT NULL,
  effective_from DATE NOT NULL,
  output_tax_account_id UUID NOT NULL REFERENCES gl_account(id),  -- liability, tax we owe
  input_tax_account_id UUID REFERENCES gl_account(id),             -- asset, recoverable — build now, activate later
  is_recoverable BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE tax_transaction_line (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  journal_line_id UUID NOT NULL REFERENCES journal_line(id),
  tax_rate_id UUID NOT NULL REFERENCES tax_rate(id),
  taxable_base_amount DECIMAL(19,4) NOT NULL,
  tax_amount DECIMAL(19,4) NOT NULL,
  direction VARCHAR(10) NOT NULL CHECK (direction IN ('OUTPUT','INPUT'))
);
```

MVP POSTING RULES TO WIRE (RULE_CODE → ACCOUNTS TOUCHED)

FLOW	RULE CODE	DEBIT	CREDIT
Invoice issued	AR_INVOICE	1120 AR	Revenue + 2150 Tax Payable
Payment received	AR_PAYMENT	1115 Undeposited Funds	1120 AR
Bank deposit	BANK_DEPOSIT	1112 Checking	1115 Undeposited Funds
Goods receipt	INV_RECEIPT	Inventory	2155 GRNI
Vendor bill	AP_BILL	2155 GRNI	2110 AP
Vendor payment	AP_PAYMENT	2110 AP	1112 Checking

TRIMMED FOR MVP

Activate only one non-recoverable tax rate for launch — the schema above is already VAT-ready via input_tax_account_id, so switching on a second jurisdiction later is configuration, not a migration. Skip three_way_match_exception — accept vendor bills as entered; add PO/receipt/bill variance checking once real purchasing volume exists.

COMMON MISTAKE

Posting revenue at invoice time and COGS at shipment time with no matching mechanism — gross margin looks wrong for any order where the two dates straddle a reporting cutoff. Keep COGS and revenue recognition in the same posting batch if your flow ships before invoicing.

DONE WHEN

A full invoice → payment cycle and a full PO → receipt → bill → payment cycle post with zero manually-keyed journal entries.

ENFORCE ON EVERY POST

MVP posting validation checklist

The posting engine (Phase 1) must reject the entire transaction — never partial-post — if any of these fail:

- ☐ **Balance:** `sum(debits) = sum(credits)` for the entry
- ☐ **Postability:** every line's account has `is_postable = TRUE` — no posting to Header accounts
- ☐ **Active:** `gl_account.is_active = TRUE`
- ☐ **Period open:** target `fiscal_period.status = OPEN`
- ☐ **Control account routing:** a line posting to an `is_control_account = TRUE` account must include `customer_id` or `vendor_id` — a posting to AR with no customer is a data-integrity time bomb for aging reports
- ☐ **Duplicate detection:** check `(source_module, source_document_type, source_document_id)` uniqueness before posting — a retried API call must never become a duplicate debit to a customer's AR

API SURFACE

Required endpoints for MVP

GET /companies/{id}/gl-accounts	list, filterable by category/type/active
POST /companies/{id}/journal-entries	create (DRAFT or PENDING_APPROVAL)
POST /companies/{id}/journal-entries/{id}/approve	requires approver $\neq$ creator
POST /companies/{id}/journal-entries/{id}/reverse	flips lines, posts new entry, flags original REVERSED
GET /companies/{id}/customers/{id}/aging	30/60/90 buckets from ar_subledger_transaction
POST /companies/{id}/ar/cash-applications	match payment to invoice(s)
GET /companies/{id}/reports/trial-balance?asOfDate=	
GET /companies/{id}/reports/balance-sheet?asOfDate=	
GET /companies/{id}/reports/income-statement? periodStart=&periodEnd=	

No PUT/PATCH/DELETE on a POSTED journal entry, ever — return 409 Conflict if attempted. This is the immutability rule made concrete in the API.

EXPLICITLY DEFERRED

Do not build these for MVP

Fully designed in the full spec, ready later — deferring costs nothing, since none require changing the MVP schema above.

DEFERRED MODULE	SPEC REF	BUILD WHEN
Multi-branch dimension	\$15	When you open a second branch
Multi-currency + FX revaluation	\$16	First foreign-currency customer or vendor
Full automated year-end closing	\$13	One careful manual entry works for year one
Fixed Assets & Depreciation	\$27	First depreciable asset you need to track
Bank Reconciliation	\$28	Before your first real month-end close
Cash Flow Statement	\$29	When investors/lenders ask for it

Budgeting	\$30	First annual planning cycle
Document Attachments	\$31	Before your first external audit
Recurring Journal Templates	\$32	When manual monthly entries get annoying
Document Numbering (gapless)	\$33	Before any jurisdiction requires it legally
Tax Filing & Statutory Reports	\$34	Before your first VAT/tax filing deadline
Dimension Reporting (cost center/project)	\$35	When you need departmental or project P&L
Approval Notifications & Escalation	\$36	Once the hardcoded Phase 4 rule isn't enough

RULE OF THUMB FOR ADDING ANY DEFERRED MODULE LATER

If it needs a new table that references `gl_account`, `journal_entry`, or `journal_line` by foreign key, it's additive and safe to bolt on. That was already checked when the full spec was designed.

Build strictly in Phase 0 → 1 → 2 → 3 → 4 → 5 order. Don't let anything from the deferred table sneak in early just because it looks small — every hour there is an hour not spent getting the MVP statements correct, and none of it is needed to launch. Full field-level detail and edge cases for everything above live in the 37-section spec.